

Task 10 — Machine Learning Theory Notes

1. What Is Machine Learning?

Machine learning is a way of getting a computer to find patterns in data and use them to make predictions, instead of being given an exact set of rules to follow. Rather than a programmer writing “if this, then that” for every possible case, the computer looks at examples and works out the pattern for itself.

There are two main types:

- Supervised learning — the computer is shown examples where the correct answer is already known.
- Unsupervised learning — the computer is only given the data, with no correct answers, and has to find structure on its own.

2. Supervised Learning

In supervised learning, every row of data comes with a known, correct answer attached (this answer is called the label or target). The model's job is to learn the relationship between the data and the label well enough to guess the label for new, unseen data. It splits into two kinds:

- Regression — the answer is a number that can fall anywhere on a scale, such as a price, a temperature, or a pollution level.
- Classification — the answer is a category, such as “spam” or “not spam”, or “high pollution” or “low pollution”.

3. Unsupervised Learning

Unsupervised learning works without any labels at all — the model is just handed the raw data and asked to find some kind of structure in it. A common example is clustering, where the goal is to group similar data points together.

KMeans is a simple and popular clustering method. It looks at how close data points are to one another and groups the ones that are close together into the same cluster, without ever being told what the “correct” groups are.

4. Data Preprocessing

Real data is rarely clean, so it needs to be prepared before a model can learn from it well. This usually means:

- Removing duplicate rows, so the same example isn't counted twice.
- Handling missing values, either by filling them in with a sensible number or removing the affected rows.
- Selecting the useful features, and dropping ones that don't help or that don't make sense to include.

- Standardizing numerical data, so that all features are on a similar scale — this matters a lot for methods like gradient descent, which can behave badly if one feature has huge numbers and another has tiny ones.

5. Train-Test Split

Before training, the dataset is split into two parts. The training set is what the model actually learns from. The test set is kept completely separate and is only used afterward, to check how well the model performs on data it has never seen before.

This split matters because a model can easily look like it's doing well simply by memorizing the data it was trained on. Testing on unseen data is the only honest way to know whether it has actually learned something useful, rather than just memorized the answers.

6. Baseline Models

A baseline model is the simplest possible model — one that barely tries at all. It exists as a sanity check: if a more advanced model can't beat this trivial one, it isn't actually learning anything useful.

- Regression baseline — simply predicts the average (mean) of all target values, every single time, no matter what the input is.
- Classification baseline — simply predicts whichever category shows up most often in the data, every single time.

7. Linear Regression

Linear regression predicts a continuous number by combining the input features in a weighted sum — in effect, it tries to draw the best-fitting straight line (or flat plane, with more features) through the data.

The model is trained using gradient descent, which gradually adjusts the weights to minimize the Mean Squared Error — the average of the squared differences between what the model predicted and what the true value actually was. Squaring the errors means that being very wrong is punished much more heavily than being only slightly wrong.

8. Logistic Regression

Despite having “regression” in the name, logistic regression is actually used for classification, most often when there are only two possible categories (binary classification).

It works by taking a weighted sum of the features, just like linear regression, and then passing that result through a sigmoid function, which squashes any number into a value between 0 and 1. That output can be read as a probability — for example, a 0.8 might mean “80% likely to belong to the positive class.”

It is trained by minimizing Binary Cross-Entropy Loss, which measures how far the predicted probability is from the actual class (0 or 1); the further off a confident wrong prediction is, the more heavily it is penalized.

9. Gradient Descent

Gradient descent is the method most of these models use to actually learn. It starts with the model's parameters (weights and bias) set randomly, then repeats a simple loop:

- Measure how wrong the current predictions are (this is the loss).
- Work out which direction to nudge each parameter to make the loss a little smaller.
- Take a small step in that direction, and repeat.

The learning rate controls how big each of these steps is. If it's too high, the model can overshoot the best answer and never settle down. If it's too low, learning happens correctly but painfully slowly, and may not finish improving in the time allowed.

10. KMeans Clustering

KMeans finds groups (clusters) in unlabeled data through a repeating four-step process:

- Randomly place K starting points, called centroids (K is the number of clusters you've chosen).
- Assign every data point to whichever centroid is closest to it.
- Recalculate each centroid as the average position of all the points assigned to it.
- Repeat the assign-and-recalculate steps until the centroids stop moving — this is called convergence.

11. Evaluation Metrics

Once a model is trained, its performance needs to be measured with numbers, not just a gut feeling. Different problem types use different metrics:

Regression

- MAE (Mean Absolute Error) — the average size of the errors, ignoring whether they're too high or too low.
- MSE (Mean Squared Error) — like MAE, but errors are squared first, so large mistakes count for much more.
- RMSE (Root Mean Squared Error) — the square root of MSE, which brings the error back to the same units as the original target, making it easier to interpret.
- R-squared — a score from roughly 0 to 1 showing how much better the model is than just guessing the average every time; higher is better.

Classification

- Accuracy — the percentage of predictions that were correct overall.
- Precision — of everything the model predicted as positive, how much actually was positive.
- Recall — of everything that was actually positive, how much the model actually managed to catch.
- F1 Score — a single number that balances precision and recall together.
- Confusion Matrix — a small table showing exactly how many predictions were correct or wrong, and in which direction.

Clustering

- Inertia — how close, on average, points are to their own cluster's centroid; lower usually means tighter clusters.
- Silhouette Score — measures whether points fit well in their own cluster compared to the next-closest cluster.
- Cluster Counts — simply how many points ended up in each cluster, which can reveal lopsided or empty groupings.

12. Data Leakage

Data leakage happens when information that the model shouldn't have access to at prediction time accidentally sneaks into training. This makes the model look far more accurate than it really is, because it's secretly being handed clues it wouldn't have in a real, honest situation.

A few common ways this happens: using a column that's really just a disguised version of the target itself; doing preprocessing steps (like scaling) on the whole dataset before splitting it, so information from the test set leaks into training; or accidentally letting test data get used during training or tuning at all.

13. Learning Outcomes

By working through this task, the goal is to genuinely understand — not just recite — how preprocessing, gradient descent, regression, classification, clustering, evaluation metrics, and visualization all fit together, by building each piece from scratch rather than only calling a library function.